Career Crox | CC870

Save & Submit repair record - 19 October 2026

Request and confirmed finding

Fix the false "Phone number is required" error even with a visible ten-digit number, restore incomplete-profile Save, and deliver the complete GitHub folder. The production browser form reproduced the failure with **7895409085** visible. An obsolete Quick Add fetch bridge redirected **/api/candidates/new/commit** to **/api/candidates**. The phone stayed inside **fields.phone**, while the wrong endpoint expected a flat phone field. This caused the false required error on both manual Save and background drafts.

Repair

Disable the obsolete bridge when React owns the profile. Independently pass atomic new-profile commit requests through unchanged, including when ownership is enabled later. Version the integration script URL as CC870 to fetch the repaired script. The backend command contract and existing profile protections are preserved.

Verified behavior

Save stores an ID-less server draft even when only a name, partial email, note, or no phone has been entered. Reload retains the same draft and values. Save & Submit validates the required details and commits the candidate, note and submission together. Short and long numeric phones are accepted without a ten-digit limit. A repeated submission keeps the same Candidate ID.

Validation	Passed
Real Chromium form, request payload and persisted data scenarios	11
Legacy bridge ownership and late-owner request routing	2
Real Express API, duplicate protection and import checks	15
Phone input, paste, display and validator checks	5
Single compact reminder popup lifecycle checks	9

Delivery and deployment

Full GITHUB folder: **96 frontend files + 96 other files**, including this report. Changed source: frontend/public/assets/js/crm-integrations.js, frontend/index.html and README_DEPLOY.md. Replace the complete repository contents from this ZIP, preserve the configured environment values, run the usual Render build/deploy, then press Ctrl+F5. No SQL reset or Android replacement is needed for this repair.

Continuity and scope

CC868 removed the fixed phone-length restriction; CC869 kept the small left reminder and removed its duplicate. Both remain covered by regression checks. This report continues the available CC869 deployment record. The production build and release hashes pass. Browser/API checks use the real app with isolated local fixture data; live Supabase and Render were not accessed. No existing production candidate data was changed.

CC871 | Pipeline Hub - Stage 1

19 October 2026 | Continuation of the CC870 work record

Confirmed cause and repair

Candidate-wide upsert, frontend/backend deduplication, PostgreSQL trigger and local store guard treated every interview as the same record. Each interview now has a separate revenue_id. Stable command receipts prevent double-click/retry duplicates. Client/process/location changes from Update start a new attempt; the old interview remains intact. General Candidate status changes cannot rewrite several interview histories.

Completed in this stage

Revenue Hub is labelled Pipeline Hub. Save keeps the editor open with Add another interview; candidate details and currently attached files are reused. Candidate Profile has Add to Pipeline Hub. Client, process and client location appear first. The interview clock runs locally in IST and freezes on Save. Removed qualification, communication, candidate experience/salary and selection/joining date inputs. Candidate email appears below the resume; an extracted different email requires a choice. Inform to Manager directly invokes the existing native resume + image + text share workflow. Duplicate share/contact controls are removed.

Shared catalog and requested defaults

Zomato, LivPure and TVS are included. A typed process can be saved for everyone; saving an interview also persists its process. Concurrent additions are merged. Client locations remain editable.

Client	Locations	Payout / days
1Point1	Gurgaon S18; Mumbai; Turbhe, Navi Mumbai; Indore	5,000 / 60; Indore 4,000
Globiva	Gurgaon Sector 18	5,000 / 60
Vertex Group	Gurgaon Sector 20	5,000 / 90
Cogent	Gurgaon S18; Noida S127	5,000 / 60
iEnergizer	Noida Sector 60	5,000 / 60
Eureka	Noida Sector 62	4,000 / 60
Alturist	Noida S127; Noida S6; Panchkula	5,000 / 60 at S127; 90 elsewhere

Razorpay process: 4,000 payout. Location interpretations: Cogent Noida Sector 127; Alturist Noida Sector 6 and 127. These can be corrected before final packaging.

SQL and verification

Safe update plus full secured/reset and two-part variants are updated. The safe migration was executed twice in the PGlite PostgreSQL engine: old rows, custom processes, targets and WhatsApp preferences survive. New terms seed once; later manual edits survive reruns. Passed: 11 actual Express API cases, 10 production Chromium form scenarios and 4 SQL cases. Production build and release hash checks pass. GitHub upload groups: frontend 96 files; remaining 97 files.

Practical limits and next stage

No live Supabase/Render deployment was performed. Native Share payload and user activation were tested; the user still selects WhatsApp and confirms Send in the operating system. Attached files are reused within the open editor, not stored by this repair. The clock adds no polling; normal explicit reads/writes still transfer data. Pending: My Drafts cards and permissions, two-hour arrival/feedback reminders, reminder styling, follow-up popup profile routing, and the Interviews screen font/layout. Final five-part delivery follows the user's next instruction after checking remaining limit.

CC872 | Filters, drafts & reminders

CAREER CROX CRM - 19 October 2026. This update continues the preceding CC870/CC871 work record. These latest pages supersede the earlier pending-work list.

1. Universal filters and ownership

Cause confirmed: the PostgreSQL candidate list supplied only its current page as the option source. The frontend also excluded TL/manager accounts from the recruiter picker. A separate role-scoped DISTINCT catalog now supplies values from every accessible candidate, independent of pagination and selected filters. It transfers values and counts, rather than full candidate profiles.

Manager: all current managers, TLs and recruiters. **TL:** self and authorized team. **Recruiter:** self. Zero-allocation employees remain selectable and correctly return zero matches. Current owner identity takes priority over stale historical recruiter codes; older name-only ownership is supported.

Candidate filters have stable numbers, searchable navigation, removable active-filter chips, exact matching counts and a location breakdown in the filter drawer. Selections are counted separately from matched profiles. Project-scoped previews retain their project constraint. Shared staff choices also feed Submissions, Interviews and Hot Leads; Pipeline Hub uses its full visible roster. Dropdown values do not change when visiting page 4. Changed CSS/JS assets now use CC872 URLs so an existing browser cache fetches the new files.

2. My Drafts and recovery

The Candidates screen has a counted My Drafts card. Manager sees all employee server drafts, TL sees team drafts, recruiter sees own drafts. Leadership can inspect another employee's draft without taking ownership. The owner can resume into the real candidate form. Drafts remain separate from candidate rows and do not allocate Candidate IDs.

New profiles use the existing ID-less autosave. Unsaved edits to existing profiles also save separately with a baseline check and serialized writes. Device drafts remain available during a network failure and retry after reconnect. A newer device copy is preferred over an older server draft; other users can see it after successful synchronization. Successful Save clears the preceding edit draft without clearing later edits.

3. Arrival, feedback and follow-up

Pipeline Hub exposes Arrived / Feedback. Arrival records a stable timestamp; feedback checks are due after two hours and repeat on local snooze/clock state. Scheduled arrival checks use the interview time in IST. Recruiter, team and manager scopes are preserved. Candidate-only arrived profiles also receive feedback reminders.

The compact follow-up card and the reminder center open the exact candidate profile. Duplicate Pipeline reminder insertion was removed. Reminder cards use consistent height and distinct category colors. Interview table typography, group headers and row spacing were normalized.

CC872 | Verification & installation

Checks completed locally

Verification	Passed
Existing Pipeline API regression	11
Existing production-browser Pipeline regression	10
Earlier safe-migration regression	4
New role, filter, draft and browser checks	16
Actual PostgreSQL-engine filters, paging and drafts	8
Actual compact-card clicks and simulated local clock	3
Total checks	52

The PostgreSQL checks execute the actual facet, owner-filter, pagination/count and draft queries in PGlite, using real bucket-policy SQL helpers. They cover page 1/page 4, rare values, zero-result filters, stale owner codes, name-only ownership and scoped draft metadata. Production Chromium checks cover numbered options, active criteria, restored form fields and exact follow-up navigation.

The reminder clock test advances five minutes without a reminder-summary network request, then advances to the two-hour feedback deadline while maintaining an active fixture session. It confirms one category-colored compact card at the agreed height. This is a simulated local clock test, not a nine-hour production endurance run.

Install the five-part package

Extract all five ZIPs into the same destination to obtain one CC872_FINAL folder. Upload the **contents** of DESKTOP_CRM/GITHUB to the repository root: frontend has **96 files**; all remaining GitHub files total **97**. Keep frontend/backend/scripts paths intact.

For an existing configured CRM database, run **DESKTOP_CRM/SUPABASE/CC872_FILTERS_DRAFTS_REMINDERS_SAFE_UPDATE.sql** before deploying this web code. It includes the preceding CC871 Pipeline migration and the additive CC872 draft/timestamp update. It was run repeatedly with existing fixture rows preserved. The full SECURED script and its two parts are updated alternatives. RESET scripts intentionally erase CRM data and are for an expressly chosen fresh setup.

LOCAL_TEST uses the current production frontend and shared backend; keep it beside DESKTOP_CRM. Node.js 24 and an initial dependency install are required. Original Hot Lead templates are retained. Android is carried forward unchanged from the preceding CC865 Android project, with its original Studio launcher and version labels.

Egress and remaining real-world checks

The five-minute reminder network fallback was removed from the main reminder owner. Scoped live signals, relevant writes, reconnect, foreground catch-up and manual refresh update its snapshot; due/snooze clocks render locally. Normal API reads, writes and live connections still transfer data. Zero total egress is not claimed.

Still unverified: live Render/Supabase deployment; a nine-hour multi-user run; actual Supabase egress measurement; Android Studio APK compilation and device behavior; delivery of messages inside external WhatsApp. Native Share opens the OS share flow, where the user chooses the recipient and confirms Send. The separate public website remains paused and is excluded from this CRM delivery.

Current test-result JSON files accompany this report in README. Earlier reports remain historical evidence; no live candidate data was accessed or modified by these local checks.

CC873 | Return to work, keep your view

CAREER CROX CRM - 20 October 2026. Continues the preceding four-page CC872 record. The current pages supersede earlier pending lists for the changes described below.

1. Filters, sorting and profile navigation

Confirmed cause: Submissions wrote its saved view state but explicitly discarded its filters and selected card when mounting. Candidates restarted on page 1, and other lists also reset local view selectors. State now belongs to the signed-in principal and current browser tab. Back and reload preserve the chosen view; an explicit URL filter still takes priority.

Candidates, Submissions, Interviews, Hot Leads, Bucket and Pipeline preserve applicable filters and sorting. Task, Client Pipeline, Follow-up and Recycle review view selectors were also checked or retained. A Back to list button on Candidate Profile returns to its original list and query. A same-origin check prevents external return destinations.

The shared sort control includes newest/oldest submission, salary high/low, communication high/low, experience high/low, name A/Z, created/updated time and interview date. Candidate and Hot Lead sorting runs in PostgreSQL before pagination, with stable identity tie-breaks. Zero remains a numeric value; blanks sort last. Salary currency/k and experience months are normalized. All 16 directions were compared between actual PostgreSQL and list sorting.

The preceding role-scoped universal catalogs and numbered Candidate filters remain. Bucket now also reads that complete catalog, including TL/manager and zero-allocation employees, instead of deriving choices from its current rows. Manager sees permitted staff globally; TL sees the authorized team; recruiter sees own data. Applied criteria, matching count and location breakdown remain available.

2. Selected cards and Android branding

Clickable summary cards use a consistent red outline and small indicator. Follow-up cards now mark their actual category. Pipeline Interview, Feedback, Selected and Joined metrics open the matching stage and preserve its selected state. Read-only financial overview metrics remain summaries.

Android now displays a rounded native Career Crox wordmark, a compact Made by Aaryansh signature and Version CC873 below it. The signature has a 2dp bounce that stops while hidden/detached and respects motion preferences. Version code is 278. Studio and preview launchers are retained with updated labels.

3. Egress reductions with current data

Hot Leads previously read up to 200 extra profiles for the dialer on each page. It now reads the visible page. Candidate fallback responses also keep the dialer tied to the displayed records. Duplicate Submissions summary fetching was removed.

Server snapshots compare current selected row values and transfer only changed rows and removals. This covers key broad/scoped table reads, Candidate/Hot Lead pages, and profile notes/history. Page order is preserved. External edits and physical removals are rechecked; caller mutation cannot poison the snapshot. Transactions bypass shared snapshots so uncommitted work is never published. Caches are bounded and access changes clear the relevant state.

CC873 | Evidence and installation

Verification completed locally

Check group	Passed
Pipeline API regression	11
Production-browser Pipeline regression	10
Safe SQL preservation regression	4
Roles, drafts, filters and browser Back/navigation	24
PostgreSQL filters, counts, paging and drafts	8
Compact reminder click and simulated clock	3
PostgreSQL sorting, snapshots and measured read workload	6
Android 35 resources, Java, DEX and preview APK signature	7
Total	73

Measured read test - not a live billing result

Fixture: 35 profiles with 2 KB notes; one cold table read, twenty unchanged reads and one changed record. Returned database JSON was **1,828,200 bytes** for equivalent full reads and **88,682 bytes** with the current snapshot path: **95.15% less**. Twenty unchanged reads returned 40 JSON bytes; one edited profile returned 2,497 bytes. A separate test coalesced twenty simultaneous identical reads into one database query.

These figures exclude PostgreSQL wire/TLS overhead, cold starts, actual PDFs, unrelated services and live user behavior. They do not prove a 95% reduction in the Supabase dashboard or a 2% daily ceiling. Newly requested/changed data and file transfers still consume egress. A daily percentage depends on the plan allowance and actual transferred bytes. No feature-blocking daily cutoff was introduced.

Browser read invalidation now follows resource dependencies, so an unrelated task write does not discard a candidate-list read. Changed static assets use CC873 cache URLs. Prior event/manual refresh and local reminder clock behavior remain; no new background polling was added.

Install the five complete ZIPs

Extract all five parts into the SAME destination to obtain CC873_FINAL. GitHub frontend has 96 source files; all other GitHub files total 97. Upload the contents of DESKTOP_CRM/GITHUB to the repository root. Keep frontend/backend/scripts paths intact. Node.js 24: npm run render-build; npm start.

For an older configured CRM database, DESKTOP_CRM/SUPABASE/CC873_SAFE_UPDATE_KEEP_DATA.sql includes the preceding Pipeline/draft/arrival upgrades and preserves data. A database already upgraded to CC872 needs no further structural migration for this release. Full SECURED SQL and its two parts remain alternatives. RESET scripts deliberately erase data and are only for an intended reset. LOCAL_TEST, Hot Lead templates and historical reports are retained. This updated PDF is in README.

Remaining real-world checks

Live Render/Supabase deployment and the nine-hour, 20-user workload remain unverified. Measure Supabase database/shared-pooler and Storage egress before and after deployment. Native resource/Java/DEX compilation and a signed preview APK were verified with local Android 35 tooling; the Android Studio Gradle build and real-phone appearance/behavior were not run. External WhatsApp delivery still requires selecting the recipient and confirming Send in the OS.

No live CRM database or deployment was changed. Current result JSON files accompany this report in README/TEST_RESULTS. Reference: Supabase, Manage Egress usage, accessed 20 Oct 2026: https://supabase.com/docs/guides/platform/manage-your-usage/egress

A portable CRM with independent recovery

The business needs three independently owned and hosted CRM instances: First, Second and Third. A manager should be able to save the whole business dataset, restore it into a prepared instance, move the team there and later bring the newer work back. Profiles, interview attempts, ownership, history, documents and pending work must remain connected.

The example that drives the design is 100 profiles on First, copied to Second, then 200 profiles after work on Second. Restoring that descendant into the older First should reconcile the original 100 and add the new 100. It must also carry changes to the original records, their interview history, files and deletion markers.

Status at this handover

Item	Verified state
Baseline application	CC873 source and prior work record available locally
New architecture	Designed and reviewed in this pack
New application code or SQL	Not started; no Multiple CRM feature is installed
Production backup or migration	Not run; no live account or database was changed
New runtime test results	None; acceptance tests here are a future execution plan

The user requested a planning checkpoint before construction. This ZIP contains that checkpoint, not a replacement application. Keep it with all five CC873 application parts. A future developer must not mistake the detailed design for working functionality.

How to read the pack

Read this document first, then the data and SQL contract, the transfer protocol, security and operations, the implementation map, and finally the verification and runbooks. Editable Markdown mirrors the six PDFs. The unchanged six-page CC873 work record is retained under HISTORY. Original requirements are under SOURCES; baseline hashes are under EVIDENCE.

Working defaults

One shared codebase and canonical schema serve all three instances. Only one instance accepts business writes at a time. Export and import require a verified manager, current password and separate four-digit PIN. A separately held offline recovery key makes a saved backup usable after the original Gmail or hosting access is lost.

Scope and non-negotiable outcomes

The business copy includes

- Candidate profiles, soft-deleted profiles, recycle history, notes, allocations and ID reservations.
- Every interview and submission attempt, including repeated interviews for the same candidate at the same client.
- Tasks, follow-ups, reminder schedules, acknowledgement and snooze state, chat history and read state.
- Employees, business accounts, roles, reporting relationships, client and process settings, drafts and operational history.
- Resumes, PDFs, Word documents, onboarding files, call recordings and other application-managed objects, with real file bytes.
- The approved application release, canonical schema recovery kit and current Architecture notes, without provider credentials.

What changes during a move

Users sign in again on the target URL. Android devices pair again and paused dialer work resumes deliberately. Instance identity, hosting URLs, API keys, session tokens, device tokens, email credentials and quota counters belong to the target environment. They are not blindly copied from the source.

Device-only edits that never reached the server are outside a server snapshot. A planned handover checks and drains phone outboxes and unsynced drafts before the source is frozen. If a device or source is inaccessible, its unsaved changes cannot be recovered by promising a complete server backup.

Boundaries that must remain visible

A full export reads data and files and therefore uses egress. Importing data does not reset a provider's quota. Three email addresses do not establish entitlement to three free hosting plans. Resource availability and the provider's current account limits must be checked during deployment.

Business data and history grow with use; permanent history cannot grow forever in a fixed-size database. The design avoids storing daily backup copies inside that database, limits temporary job payloads and deduplicates identical file content. It does not delete old activity to create an artificial size guarantee.

Definition of a safe result

The archive is complete only after all required table and file coverage checks pass, the saved local file is verified, and a restore drill proves the result. No source instance is automatically destroyed, reset or deleted. A disaster recovery from an older offline backup has a recovery point limited to that backup's committed cutoff.

Baseline audit: useful foundations

The audit used the local CC873 release. It inspected the existing backup controller, codec, compatibility mapper, routes, request limits, data store, backup page and layout integration. It did not inspect a signed-in live Supabase project.

File relative to GITHUB	Existing responsibility
backend/src/controllers/backupController.js	Snapshot, preflight, export, validation and restore orchestration
backend/src/lib/backupCodec.js	Gzip, password-derived key, AES-GCM envelope
backend/src/lib/backupRestoreCompat.js	Legacy user/schema compatibility and alias handling
backend/src/lib/store.js	PostgreSQL access, table allowlist and transaction helpers
backend/src/routes/index.js	Backup endpoints and middleware integration
backend/src/app.js	Request parsing and size limits
frontend/src/pages/DataBackupPage.jsx	Existing backup UI, local preview and submission
frontend/src/components/Layout.jsx	Navigation and application-level integration

Existing safeguards worth retaining

Manager/admin gates reject impersonated or degraded authentication for full database operations. Snapshot reads use one repeatable-read database transaction with keyset batches. The current archive has authenticated encryption. Restore validates primary keys and uses transactional writes, advisory locks and command receipts to reduce repeated application of a request.

There is already a distinction between candidate workbook export and a whole-database backup. That distinction must survive: a candidate workbook is not disaster recovery for users, tasks, settings, recordings and all remaining business data.

Evidence boundaries

Eight audited files were hashed before planning. Their original hashes are in EVIDENCE/BASELINE_AUDIT_SHA256.json. The final planning QA compares them again. The historical CC873 work record reports 73 local checks for that prior release. Those results do not prove any new three-instance migration, offline key, streaming archive or PIN gate.

The backend currently declares Node 24.x, Express 4 and pg 8. The frontend declares React 18, React Router 6 and Vite 5. Package version strings remain 1.0.863 in the inspected manifests; release labels and package versions therefore need explicit alignment during the implementation release.

Baseline audit: gaps to close before release

ID	Finding from source review	Planned correction
B01	Attachment scan covers selected /generated/ paths, not all Storage objects	Managed-asset inventory plus streamed bytes and checksums
B02	Whole payload is assembled in memory	Chunked protocol, backpressure and bounded processing
B03	Export plain limit and import ciphertext/body limits differ	One negotiated, tested size policy for every stage
B04	Active revenue validation rejects multiple rows per candidate	Unique attempt identity; repeated candidate attempts allowed
B05	PK upsert can replace newer target rows with an older snapshot	Dataset identity, lineage, revisions and explicit conflicts
B06	Legacy roster mapper relies on usernames and alias rewrites	Stable employee UUID mapping and typed reference fields
B07	Four-digit PIN and three-instance operating mode are absent	Server-enforced operation grants and instance control state
B08	Only five runtime tables are explicitly excluded	Classify all 80 tables plus future tables and mixed keys
B09	Sequence high-water reads occur after snapshot commit	Freeze-aware capture; advance only, never rewind
B10	Local files and DB writes are not one atomic service	Immutable file staging, DB commit point and undo runbook
B11	Preflight has estimates, not complete file integrity proof	Coverage, hashes, references and post-restore checks

Current file limits include 16 MiB per local attachment, 32 MiB total attachments and 1,024 files. Database limits include 100,000 rows per table and one million rows overall. The codec allows 256 MiB plaintext but rejects ciphertext above 64 MiB; the full restore request has a separate 90 MiB JSON limit. A valid export can therefore exceed a later import stage when data compresses poorly.

These are code-review findings and design targets. They have not been fixed in this planning ZIP. Other existing application behavior is not being rewritten as an incidental planning task.

Compatibility promise to implement

Keep a read-only legacy v1 import adapter, validate its content with v2 rules, and show missing-coverage warnings. An incomplete old backup cannot be relabelled a complete v2 backup. All old full-backup endpoints must share the new authorization gate; leaving one older unprotected route would bypass the feature.

Decisions, tradeoffs and unresolved measurements

Decision	Benefit	Cost or limit
Three isolated environments, one schema	Recovery does not depend on one provider login	Three setups and independent credential maintenance
Single active writer	Predictable round trips and simpler ownership	Team must follow a controlled switch procedure
Stable UUIDs plus lineage	Avoids C-number and username mismatches	One-time backfill and migration tests are required
Self-contained encrypted archive	Can restore without the original account	Full first export and local key custody are necessary
Stage, preview and verify before activation	Prevents silent stale overwrites and broken file links	Target needs temporary space and an operator checkpoint
Manager reminder after verified download	Prompts actual recoverability	Download acknowledgement needs more than an HTTP success

Measurements deferred to construction

Record actual database and object sizes, slowest file access, available Render RAM/temp space, database free space and WAL headroom, browser support and upload timeout constraints. Set tested chunk sizes and concurrency from these observations. They are not reliable constants to invent in a planning document.

A target of zero egress, or a guaranteed two-percent daily ceiling regardless of team activity, is not technically established. The design keeps standby instances quiet, avoids repeated full reads, offers local archive refresh after the core workflow is proven, and records measured bytes for each operation.

Changes to the plan during review

The design distinguishes a portable dataset from an environment. It explicitly separates durable business history from live sessions, handles repeated interviews as independent attempts, and keeps source-offline recovery dependent on an offline private key. It rejects using timestamps alone to select a winner, and it rejects treating an absent row as a deletion instruction.

An online distributed lock across three unrelated hosts is not assumed. If operators allow isolated copies to keep writing, the system can detect divergent branches during reconciliation, but cannot guarantee that both hosts were prevented from accepting all work without a shared coordinator.

Next authorized checkpoint

The structure and review pack are the output of this turn. Application construction starts only in the subsequent implementation phase. The order and completion gates appear in document 05; every feature remains marked planned until its implementation and evidence are recorded.

One dataset, three environments

Identity layers

Identity	Travels with the archive?	Meaning
dataset_uuid	Yes	The business dataset being continued
sync_uid / globally unique entity PK	Yes	The same candidate, employee, attempt or task everywhere
Display ID, e.g. C001	Yes	Stable human-facing code; not a cross-branch merge key
revision + origin branch	Yes	Ordered business change identity within a known lineage
instance_uuid and slot	No	This particular hosting environment: First, Second or Third
write epoch and accepted head	Validated control state	Which branch/head this instance may continue

Legacy rows acquire stable UUIDs once on the original dataset before its first v2 copy. Migration reruns must preserve those UUIDs. New entities receive random UUIDs; a future candidate UID must not be derived only from its displayed C-number because independent branches can allocate that number to different people.

An existing UUID primary key can serve as sync identity after the column and all references are verified. Do not add redundant identities to every table without review. Composite-key and append-only tables need explicit catalog entries describing their stable identity and canonical ordering.

Relationships and employee continuity

Every typed ownership field and relationship follows the same entity mapping. Restore keeps employee IDs, business roles, teams and recruitment ownership connected. Similar names or equal usernames are not sufficient proof that two employee records are the same person.

The target keeps a separate instance-local recovery owner with manager capabilities so an import cannot lock out its operator. It is not silently added to the recruiter roster or counted as an employee. Imported employee accounts retain their valid business identity and approved password hash format, but all users authenticate anew on the target.

Interview attempts

Candidate A can have ten interview attempts, including multiple attempts at the same client. The attempt UUID/revenue_id is unique; candidate_id alone is not unique in the attempt table. Restore validates the candidate reference and each attempt identity, then preserves the full sequence of attempts and client history.

Revisions, deletions and conflict rules

Commit-safe change tracking

Mutable durable entities carry a revision and branch origin, or an equivalent catalogued revision record. A compact journal stores identifiers, operations and committed revision metadata rather than another full copy of every row. Append-only histories reuse their existing globally unique identities where valid.

To avoid skipping a transaction that commits late, a delta cursor cannot be based only on max(sequence), wall-clock time or allocated IDs. Planned database writes acquire a dataset revision guard before business row locks and keep it to commit. Statement-level guards cover direct SQL writers; one transaction's affected rows share a committed change boundary. Deadlocks and retries must be tested. The snapshot sees a consistent head with no later-committing earlier revision omitted.

Soft-delete state, recycle reasons and durable deletion tombstones travel with the dataset. The journal preserves the identity needed to prevent deleted records reappearing on import. Missing rows in an archive are not interpreted as instructions to delete target data.

Situation	Default action
Same archive, same accepted head	Idempotent no-op; report already applied
Target is known ancestor; source is newer descendant	Apply validated changes, preserving identity
Source is older than target	Block stale import; do not overwrite silently
Both changed after their common baseline	Conflict preview; no automatic last-write-wins
Different dataset UUID	Reject normal continuation; separate deliberate migration needed
Same displayed C101, different entity UUIDs	Identity conflict; no phone/name auto-merge
Target-only rows or unexpected schema	Block or explicitly reconcile before commit

Conflict resolution is a reviewed import plan for affected records and references, not a button that merges arbitrary text automatically. Preserve the original archive and both branch heads. A conflict action invalidates the earlier preview and requires a new validated plan.

Typed configuration

Settings and feature flags are classified by table and key. Business client/process configuration is portable. API credentials, provider URLs, local recipient keys and instance policy stay local. Rebind known structured asset references through an asset map; never search-replace arbitrary names, URLs or free-text notes.

Sequences, migrations and three SQL folders

Displayed IDs, primary-key constraints, foreign keys, unique indexes and reservation registries must agree after restoration. Sequence operations are not ordinary transactional row changes; a rollback may leave a gap. A gap is acceptable. Reusing an old candidate ID or moving a sequence backwards is not.

For every generated ID source, set the next allocation above the maximum of its current safe position, the imported high-water mark, all live IDs, and reserved/deleted IDs recorded in the registry. Capture high-water information under a planned write freeze. Verify the next created candidate and related records in each target test. Do not assume all IDs are simple integers or every sequence has increment one without catalog checks.

Planned deliverable structure

Folder under SUPABASE	Purpose
FOR_FIRST_CRM	Safe upgrade of the existing business database; keep all data
FOR_SECOND_CRM	Bootstrap an empty standby; local recovery owner only
FOR_THIRD_CRM	Bootstrap the third empty standby with its own identity

Each folder receives READ_FIRST instructions, its ordered safe SQL entry point and, where length requires it, two equivalent ordered parts. The canonical schema hash is the same across all three. Bootstrap slot/instance values differ; table layouts and business constraints do not become three forks.

Migrations are additive and rerunnable. Backfill UUIDs once, verify duplicate/invalid legacy identities before enforcing new unique constraints, and inspect actual types/defaults/extensions instead of assuming a fresh database. Detect partial previous migrations and continue from explicit version markers. A fresh empty standby must not seed duplicate real employees or candidates.

Reset SQL, if retained from the older release, remains separate and clearly destructive. It is not the normal upgrade or transfer path and must never run automatically during restore. The archive contains approved schema version information and a recovery kit, not SQL that the server executes on trust from an upload.

Planned control schema

crm_control contains instance identity/mode, accepted dataset and head, local backup policy, transfer jobs, verified backup receipts and conflict metadata. It stores hashes, cursors and status, not daily archives or repeated full business payloads. Business idempotency receipts remain portable; export grants and restore-job receipts remain local.

Table coverage rules

The following pages classify all 80 table names in CC873's store allowlist. This is a planning catalog based on source inspection, not a proof that a live database contains only these objects. Phase 0 must compare pg_catalog, application settings keys, managed buckets and actual references against it.

Code	Planned treatment
P	Portable durable business state, including historical records
H	Hybrid: preserve business/history fields; normalize only explicit runtime/secret fields
L	Local runtime, credentials or environment accounting; not restored as business state

Unknown durable tables, unknown settings keys and unknown application-managed buckets block the label Complete until classified. The exporter must not silently skip them because they were absent from an old allowlist. The table registry must include primary keys, dependencies, identity mapping, sensitive fields, asset references and a schema fingerprint.

Hybrid examples that need field-level rules

Presence keeps historical attendance and accumulated totals. On a planned switch, the old open interval is captured to the frozen cutoff; the new live interval starts on login. Do not invent hours or wipe attendance history.

Dialer history and queue order travel, but execution is paused. Restored jobs cannot automatically call numbers or resend external messages. Business notifications and acknowledgements are kept; old queued delivery jobs are not replayed. Open breaks, approvals, signed URLs and provider request tokens need explicit state policies, not a broad regex that clears every similarly named field.

The runtime exclusion list does not authorize losing historical facts stored in a hybrid table. Conversely, putting a history label on a table does not authorize cloning authentication tokens. The implementation must test the exact columns and JSON keys, including new ones added later.

Catalog maintenance rule

Every future migration that adds a durable table, file type or configuration key also updates the portable-data registry, the restore dependency graph, an appropriate fixture and these documents. CI should fail a schema drift check when a durable object has no decision. Source orphans and missing files appear in coverage, rather than being quietly omitted.

Table catalog 01-12

Codes: P portable, H hybrid field-level policy, L environment/runtime. All entries are planned; confirm columns, keys and references against the actual schema in phase 0.

No. / table	Policy	Restore/export rule
01. crm_candidate_audit	P	Preserve durable business rows, stable identity, references and historical state.
02. crm_candidate_phone_claims	P	Preserve durable business rows, stable identity, references and historical state.
03. crm_candidate_id_registry	P	Preserve durable business rows, stable identity, references and historical state.
04. crm_candidate_drafts	P	Preserve durable business rows, stable identity, references and historical state.
05. crm_command_receipts	H	Keep business submission idempotency; omit local export/restore grant/job receipts.
06. crm_candidate_submit_failures	P	Preserve durable business rows, stable identity, references and historical state.
07. chat_thread_read_state	P	Preserve durable business rows, stable identity, references and historical state.
08. chat_notification_jobs	L	Queued delivery runtime; recreate safely from logical receipts, never replay history.
09. generated_downloads	L	Ephemeral links/files; recreate on demand and expire locally.
10. candidate_reset_archive	P	Preserve durable business rows, stable identity, references and historical state.
11. users	H	Keep UUID, roster, role, team and compatible password hashes; remove session/reset/API tokens.
12. candidates	P	Preserve durable business rows, stable identity, references and historical state.

A table-level label is a coverage decision, not a complete column adapter. Before coding, record its actual PK/type, dependencies, JSON identity fields and owned file references. A future unclassified durable table blocks a Complete archive.

Table catalog 13-24

Codes: P portable, H hybrid field-level policy, L environment/runtime. All entries are planned; confirm columns, keys and references against the actual schema in phase 0.

No. / table	Policy	Restore/export rule
13. tasks	P	Preserve durable business rows, stable identity, references and historical state.
14. notifications	H	Keep logical inbox, read/ack and due state; do not replay old external deliveries.
15. jd_master	P	Preserve durable business rows, stable identity, references and historical state.
16. jd_documents	P	Preserve durable business rows, stable identity, references and historical state.
17. notes	P	Preserve durable business rows, stable identity, references and historical state.
18. messages	P	Preserve durable business rows, stable identity, references and historical state.
19. interviews	P	Preserve durable business rows, stable identity, references and historical state.
20. submissions	P	Preserve durable business rows, stable identity, references and historical state.
21. active_sessions	L	Authentication sessions; never clone login tokens.
22. presence	H	Keep historical totals; close planned old interval at cutoff, start target live interval on login.
23. unlock_requests	H	Keep approval decisions/history; strip any one-use authorization artifacts.
24. activity_log	P	Preserve durable business rows, stable identity, references and historical state.

A table-level label is a coverage decision, not a complete column adapter. Before coding, record its actual PK/type, dependencies, JSON identity fields and owned file references. A future unclassified durable table blocks a Complete archive.

Table catalog 25-36

Codes: P portable, H hybrid field-level policy, L environment/runtime. All entries are planned; confirm columns, keys and references against the actual schema in phase 0.

No. / table	Policy	Restore/export rule
25. scheduled_reports	H	Keep schedule/business recipients; local provider credentials and dispatch receipts reviewed.
26. break_sessions	H	Keep history; explicitly capture/mark an open break at handover.
27. report_summaries	H	Keep historical summaries; invalidate/recompute current derived cards by version.
28. aaria_queue	H	Preserve business queue/work state; pause any executable external jobs until reviewed.
29. client_pipeline	P	Preserve durable business rows, stable identity, references and historical state.
30. client_requirements	P	Preserve durable business rows, stable identity, references and historical state.
31. revenue_entries	P	Preserve durable business rows, stable identity, references and historical state.
32. chat_groups	P	Preserve durable business rows, stable identity, references and historical state.
33. chat_group_members	P	Preserve durable business rows, stable identity, references and historical state.
34. chat_user_state	H	Preserve unread/mute/business preferences; strip live socket/session runtime.
35. settings	H	Per-key registry: portable business presets versus local instance secrets and URLs.
36. learning_progress	P	Preserve durable business rows, stable identity, references and historical state.

A table-level label is a coverage decision, not a complete column adapter. Before coding, record its actual PK/type, dependencies, JSON identity fields and owned file references. A future unclassified durable table blocks a Complete archive.

Table catalog 37-48

Codes: P portable, H hybrid field-level policy, L environment/runtime. All entries are planned; confirm columns, keys and references against the actual schema in phase 0.

No. / table	Policy	Restore/export rule
37. suggested_videos	P	Preserve durable business rows, stable identity, references and historical state.
38. interview_remove_requests	P	Preserve durable business rows, stable identity, references and historical state.
39. candidate_jd_feedback	P	Preserve durable business rows, stable identity, references and historical state.
40. revenue_hub_entries	P	Preserve durable business rows, stable identity, references and historical state.
41. mail_templates	P	Preserve durable business rows, stable identity, references and historical state.
42. mail_drafts	P	Preserve durable business rows, stable identity, references and historical state.
43. mail_logs	H	Keep historical outcomes; no resending and no stored provider tokens.
44. candidate_files	P	Preserve durable business rows, stable identity, references and historical state.
45. yt_hub_playlists	P	Preserve durable business rows, stable identity, references and historical state.
46. yt_hub_videos	P	Preserve durable business rows, stable identity, references and historical state.
47. important_resources	P	Preserve durable business rows, stable identity, references and historical state.
48. user_onboarding_requests	H	Keep onboarding decision state; strip OTP/invitation secrets.

A table-level label is a coverage decision, not a complete column adapter. Before coding, record its actual PK/type, dependencies, JSON identity fields and owned file references. A future unclassified durable table blocks a Complete archive.

Table catalog 49-60

Codes: P portable, H hybrid field-level policy, L environment/runtime. All entries are planned; confirm columns, keys and references against the actual schema in phase 0.

No. / table	Policy	Restore/export rule
49. user_onboarding_documents	P	Preserve durable business rows, stable identity, references and historical state.
50. password_reset_requests	L	Recovery tokens/OTP runtime; do not clone.
51. hot_leads	P	Preserve durable business rows, stable identity, references and historical state.
52. bda_leads	P	Preserve durable business rows, stable identity, references and historical state.
53. bda_activities	P	Preserve durable business rows, stable identity, references and historical state.
54. hr_employees	P	Preserve durable business rows, stable identity, references and historical state.
55. hr_attendance_logs	P	Preserve durable business rows, stable identity, references and historical state.
56. hr_worklogs	P	Preserve durable business rows, stable identity, references and historical state.
57. hr_leave_logs	P	Preserve durable business rows, stable identity, references and historical state.
58. hr_stage_events	P	Preserve durable business rows, stable identity, references and historical state.
59. hr_document_templates	P	Preserve durable business rows, stable identity, references and historical state.
60. hr_document_runs	H	Keep generation history and owned files; signed download URLs regenerated.

A table-level label is a coverage decision, not a complete column adapter. Before coding, record its actual PK/type, dependencies, JSON identity fields and owned file references. A future unclassified durable table blocks a Complete archive.

Table catalog 61-72

Codes: P portable, H hybrid field-level policy, L environment/runtime. All entries are planned; confirm columns, keys and references against the actual schema in phase 0.

No. / table	Policy	Restore/export rule
61. goal_post_targets	P	Preserve durable business rows, stable identity, references and historical state.
62. employee_daily_stats	P	Preserve durable business rows, stable identity, references and historical state.
63. bulk_upload_history	P	Preserve durable business rows, stable identity, references and historical state.
64. mobile_devices	L	Pairing/device authorization; pair target devices again.
65. dialer_sessions	H	Keep historical facts and queue relation; imported active work is paused.
66. dialer_queue_items	H	Preserve ordering and results; prevent automatic call replay.
67. manual_dialer_calls	H	Preserve call history; strip connection/pairing artifacts if present.
68. call_logs	P	Preserve durable business rows, stable identity, references and historical state.
69. call_quality_events	P	Preserve durable business rows, stable identity, references and historical state.
70. recording_files	H	Preserve metadata plus actual object bytes; regenerate target URLs/access.
71. live_call_state	L	Current live connection state; rebuild from target events.
72. team_chat_messages	P	Preserve durable business rows, stable identity, references and historical state.

A table-level label is a coverage decision, not a complete column adapter. Before coding, record its actual PK/type, dependencies, JSON identity fields and owned file references. A future unclassified durable table blocks a Complete archive.

Table catalog 73-80

Codes: P portable, H hybrid field-level policy, L environment/runtime. All entries are planned; confirm columns, keys and references against the actual schema in phase 0.

No. / table	Policy	Restore/export rule
73. team_chat_threads	P	Preserve durable business rows, stable identity, references and historical state.
74. egress_budget_daily	L	Provider/environment usage accounting; do not replace target quota state.
75. employee_locations	H	Keep authorized historical location records; exclude live transport/session artifacts.
76. candidate_file_contents	P	Preserve durable business rows, stable identity, references and historical state.
77. mobile_app_releases	H	Keep release metadata and owned binaries; deployment URLs rebound by typed policy.
78. crm_feature_flags	H	Per-key portable/local split; do not override target security/mode flags.
79. app_feature_flags	H	Per-key portable/local split and schema-version validation.
80. app_settings	H	Per-key portable/local split; provider credentials and instance policy remain local.

A table-level label is a coverage decision, not a complete column adapter. Before coding, record its actual PK/type, dependencies, JSON identity fields and owned file references. A future unclassified durable table blocks a Complete archive.

Archive v2: what a saved file contains

The manager downloads one self-contained .ccbackup file. It contains authenticated metadata, compressed encrypted database chunks, a deduplicated file section and an approved release/recovery kit. It is not a zip of unprotected JSON and it is not merely a collection of links to the old host.

Required manifest fields

- Format version, application release, canonical schema version and schema hash.
- Backup UUID, dataset UUID, source instance UUID, slot, branch/epoch and parent/accepted head.
- Consistent committed cutoff, UTC timestamp and the corresponding displayed office time.
- Tables, stable identity/PK definitions, canonical serialization rules, row counts and per-chunk hashes.
- File logical IDs, object versions/locators, media type, size, SHA-256 and references that use the file.
- Explicit included, excluded, missing and externally referenced coverage, plus key fingerprints.
- Recovery-kit version/hash, signature and an authenticated final trailer covering the ordered contents.

Identical file bytes appear once per archive, indexed by content hash. Structured file locators are portable and are rebound to target storage after verification. Original filenames are metadata, not executable paths. Reject path traversal, unbounded nesting and unapproved archive members.

Completeness of files

Collect application-owned local generated files, byte-bearing database records and all registered application-managed Storage buckets, including their inventory of owned objects. Candidate resumes, onboarding documents and call recordings require byte coverage, not just SQL rows. Orphan objects are reported and included if they fall inside the declared owned bucket scope.

External third-party links are explicitly listed as external dependencies. The system cannot promise control of an outside host. Missing required owned bytes or unreadable objects prevent a Complete result. A file with the right name but different bytes must not pass a checksum check.

Supabase's normal database backup does not include the actual Storage object bytes. The separate file pipeline is therefore part of the recovery design, not an optional UI enhancement. See the official source register in document 06.

Export: one consistent snapshot and a verified file

1. Verify manager capability and fresh password plus PIN. Create an operation-bound, one-use grant. Show estimated rows, files, bytes, capacity and coverage limits before starting expensive reads.
2. For a planned switch, freeze source writes and drain inflight work first. For a routine daily backup, identify a repeatable-read committed cutoff and mark that later work is outside this archive.
3. Read the database on one repeatable-read connection using bounded keyset/cursor batches. Capture schema and committed head with the snapshot. Register the file versions referenced by that same state.
4. Stream immutable file versions, validate hashes and coverage, compress/encrypt bounded chunks and write them with backpressure. Do not create one giant JSON object in Node or one giant browser Blob.
5. Finalize the ordered manifest and authenticated trailer only after every required object succeeds. A partially received file has no valid complete trailer and cannot be imported.
6. Close the local file, verify its saved bytes/hash and send a receipt bound to the completed job and cutoff. Only this verified receipt refreshes the backup reminder.

Snapshot and file consistency

SQL MVCC does not freeze an object-storage bucket. The application must use immutable/versioned objects, preserve pinned versions while the export is in progress and prevent a file delete from invalidating an active snapshot. The safe default is a new content-addressed object on replacement, followed by later reference-aware garbage collection.

If a connection or export fails, the full export restarts from a fresh snapshot unless the original snapshot and immutable staged chunks are still demonstrably valid. Never resume arbitrary new database reads into fragments of an older cutoff. The UI may resume a verified restore upload of the same immutable archive independently.

Browser behavior

Use streaming file save and a worker where supported. The fallback download path must be tested at the supported maximum size and must read back the saved file before marking it verified. Provide useful progress: tables, files, bytes, cutoff and failed item. Progress messages do not contain database row content or credentials.

Import: stage, preview, commit, verify, activate

1. Authenticate on the target, check password and PIN, and inspect the archive header. The offline key decrypts the data key locally in the manager's browser; the offline private key is never uploaded.
2. Upload bounded authenticated/decrypted chunks over TLS into private job staging. Resume only when job, archive hash and chunk identity match. Authenticate each chunk before accepting its contents.
3. Verify all hashes, final trailer, trusted source signature, format/schema, dataset, lineage, table coverage, row identities, relations, asset availability and resource headroom.
4. Build a diff preview: inserts, changes, unchanged records, tombstones, target-only records, conflicts, files and bytes. Bind that preview to the backup hash, target accepted head, policy version and mapping decisions.
5. On a fresh manager commit authorization, fence target writes and pause schedulers. Check that the target still has the previewed head. If anything changed, return a stale-preview error and recompute; do not commit a different plan.
6. Create the encrypted pre-restore undo checkpoint, apply rows in dependency order in one database transaction, rebind typed assets, preserve local recovery access and advance ID generators safely.
7. Verify references, ownership and logical counts, commit, invalidate caches/access epochs, then run file and UI checks while the target stays read-only. Promote only after these checks pass.

Commit boundary

There is no single atomic transaction across PostgreSQL, local disk and object storage. New files are staged under immutable names before their database references become live. The database commit is the reference switch. If the DB transaction fails, old rows remain and only new unreferenced staged objects are eligible for cleanup.

If verification fails after commit, keep the target read-only and use the validated undo checkpoint or a separately verified pre-restore archive. Do not blindly delete staged objects that now have live references. Promotion is its own recorded operation, and the source is not deleted.

The preview may never hide a target-only business row to make counts look equal. Resolve the branch mismatch explicitly before the target can become active.

A 100-to-200 round trip without duplicate profiles

Known descendant case

First exports head H1 with 100 candidate UUIDs. Second accepts H1, becomes the only writer and creates 100 additional candidates while updating some of the first 100. Its next archive is H2, a known descendant of H1. First is still frozen at H1.

The import matches the first 100 by UUID, retains their relationships, applies the H1-to-H2 changes and adds the new 100. Deleted profiles remain represented by their soft-delete state or tombstones. Reapplying H2 returns Already applied. Interview counts use attempts, so five interviews for one person remain five records.

Divergence case

If First also changed after H1, neither branch wins just because its file has a later timestamp. The preview reports the two branch heads and the affected entities. No automatic overwrite runs. A reviewed reconciliation may accept one side or map specific records and references, then creates a new explicit head with an audit trail.

Stale and unrelated archives

H1 cannot silently overwrite H2. A backup from another dataset UUID is rejected as a continuation. A displayed C101 shared by different candidate UUIDs is a collision, even if both rows have a plausible phone number. A matching phone is not authority to erase a separate candidate.

Logical equivalence checks

Compare canonical identities, business fields, foreign-key links and content hashes. Runtime tokens, instance URLs and current live-online status are intentional exceptions. Record both the exceptions and their reason in the restore report. Counts of due reminders can naturally advance as time passes after the saved cutoff; the saved due dates and acknowledgement history remain unchanged.

Business command receipts follow the dataset so already accepted candidate submissions are not replayed. Restore/export operation grants and local delivery queues do not travel. Outbound calls, emails and chat pushes require logical event deduplication; restoring old history must not send it again.

Lower-cost refresh of an existing local archive

The first reliable feature is a full streaming backup and restore. An additional phase can reduce repeated source reads: Refresh existing local backup. It is implemented only after the full workflow and commit-safe revision tracking pass their tests.

The manager supplies the previous verified full archive and the offline recovery key in the browser. Its signed source checkpoint, dataset and lineage are validated. The source returns only committed changed rows, deletion markers and new/changed file objects after that checkpoint. The browser merges them with the validated local base and writes a new self-contained archive with fresh encryption.

The result must restore on an empty target without the old base file. A standalone delta is never labelled Whole CRM. If the base is missing, corrupted, unrelated, too old for the retained cursor horizon or missing a required file, the UI falls back to a complete source export.

Incremental correctness requirements

- Use a commit-consistent change boundary; do not skip late-committing transactions by advancing a naive sequence cursor.
- Carry deletions, relationship changes and employee/team updates as well as new rows.
- Keep required old file bytes from the local base, deduplicate new bytes and remove only references that the validated new state actually drops.
- Authenticate the new final manifest and source delta proof. Do not let an edited local base impersonate a server-approved checkpoint.
- Re-encrypt with a new data key; avoid nonce reuse and mixed ciphertext from different archive identities.
- Verify the final local file before refreshing the reminder clock.

Space and egress tradeoff

This avoids re-reading unchanged source content but uses local disk/CPU and still reads new data. A full archive remains proportionate to the retained dataset. The business database does not store a daily archive or a full historical row clone to enable this optimization.

Measure bytes transferred by the database, Storage, backend and browser separately. Restoring creates target storage and write activity; it does not erase earlier source egress. Keep standby CRM polling and reminder workers disabled throughout normal use.

Failure handling and compatibility

Failure point	Required behavior
Wrong key, bad signature, malformed header	Reject before business import; no active data changes
File missing or hash mismatch	Incomplete job; no Complete label and no commit
Network interrupted during export	Keep prior verified file valid; restart a new consistent export
Restore upload interrupted	Resume the same archive/job within policy; reject mismatched chunks
Target changes after preview	Reject stale preview; rebuild diff and require fresh authorization
SQL constraint or identity failure	Roll back rows; retain useful error details without secret payloads
Commit succeeds, UI/file verification fails	Keep read-only; investigate or restore verified undo checkpoint
Old source returns after disaster switch	Quarantine/read-only before team work or reconciliation

Legacy import adapter

The existing career-crox-backup v1 format remains readable through an isolated adapter. Decrypt and validate with legacy limits first, then convert into the canonical staged model. Do not execute archived SQL or reuse a legacy route that bypasses PIN and capability checks.

Old archives may lack real Storage bytes, dataset UUIDs or full lineage. The preview must state those limitations. Onboarding a legacy dataset requires a deliberate identity adoption step and a target inventory; it cannot confidently infer a shared ancestor from matching names or record counts.

Staging retention

Use private temporary files or a private object prefix for job payloads. Encrypt persisted staging with an instance-local key and enforce short leases. Never place database rows or private archive bytes at a public download URL. A crashed job can be resumed or expired according to its checkpoint; cleanup must check both job state and live references.

Keep a small permanent audit of operations, manifest roots, actor, cutoff, result and exceptions. Expire bulky transient progress/chunk metadata after the recovery window. Retention of business activity, tombstones and legally needed history is a separate policy and is not shortened to save temporary job space.

Release evidence includes the exact file inventory, missing/external coverage, before/after logical hashes and actual rollback behavior. A successful HTTP response is not sufficient proof of a recoverable archive.

Manager authorization and credential boundaries

Only a verified manager or the explicitly configured recovery owner can export or restore the full CRM. TLs and recruiters have no such capability. Impersonation, degraded authentication and a forged role field cannot authorize the operation.

The current manager password and a separately configured four-digit PIN are required. PIN verification runs on the server, with a salted scrypt verifier and an instance-held pepper. Use a fresh CRM PIN. Five failed attempts cause a planned fifteen-minute cooldown, with per-account and per-origin controls; implementation must avoid an easy account-wide denial of service.

Operation grants

After verification, issue a short-lived, one-use grant bound to the actor, session, instance, operation and job. Restore commit also binds the archive hash and preview hash. Enforce CSRF protection where cookie authentication is used. A preview authorization cannot be replayed for a different file or later commit.

The old full-backup endpoints must use the same gate or be disabled. A new secure screen cannot compensate for an older API that still downloads the whole database with only a session cookie. Candidate workbook export keeps its own scope and permission contract and is visibly distinct.

Local versus portable secrets

Provider service keys, database connection strings, JWT secrets, SMTP/API credentials, local PIN verifier/pepper, recovery-owner credentials and instance signing private keys stay with the environment. They are excluded from the portable business package.

Business user password hashes can be included in the encrypted archive when the installed credential format is compatible, so restored employees retain their accounts. Never export plaintext passwords. Active sessions, reset tokens, OTPs, phone pairing tokens and device authorization are not cloned. Users sign in again and phones pair again on the new host.

Audit export, preview, commit, promotion, rollback, PIN change and failed authorization with time, actor, job and outcome. Do not log passwords, PINs, private keys, decrypted rows or complete archive bodies.

Offline recovery and authenticated encryption

Password plus PIN authorizes a CRM operation. It is not sufficient protection for a lost backup file: a four-digit PIN has only 10,000 possibilities. The archive uses an independent cryptographic recovery key, so it remains usable after loss of the original Gmail or source hosting credentials.

Planned key flow

Generate an RSA-3072 OAEP/SHA-256 recovery key pair in the manager browser. Register only its public key with each prepared instance. Save the private recovery-key file in two independent offline locations. It is not inside the backup and is never stored in the source Gmail, database or hosting environment.

For each export, create a fresh random AES-256 data key. Compress bounded chunks and encrypt them with AES-GCM. Wrap the data key with the registered recovery public key. On restore, the browser unwraps it using the offline private key and validates/decrypts chunks locally before sending bounded data to the target over TLS.

Use unique 96-bit nonces per data key, a 128-bit authentication tag, and authenticated metadata covering archive identity, chunk type/index and declared lengths. Verify the tag before consuming a chunk. An authenticated final trailer covers the ordered content so truncation, reordering or substitution cannot be accepted as a complete archive.

Trust in the source

Sign a canonical logical checkpoint with the source instance's Ed25519 key: dataset/head, schema, cutoff and table/file content roots. The encrypted final manifest binds that proof to the actual archive. A refreshed/re-encrypted local archive must recompute the same roots; the source never blindly signs a browser-supplied digest. Pin signer fingerprints on standby setup and in independent recovery notes. Unknown or replaced signing identities require trust review.

Node crypto and browser Web Crypto are the proposed standard primitives. Interoperability, negative tests and memory limits must pass before deployment. This plan is not a cryptographic implementation review.

Losing all copies of the offline private key makes these encrypted backups unrecoverable. A reset of the CRM password or PIN does not decrypt them. Possession of the key and a backup exposes its business contents, so offline key custody is part of the operating procedure. A compromised live source can still export altered business data; retain older verified offline copies.

Switching First, Second and Third

Instance modes

Mode	Business writes	Background activity
ACTIVE	Allowed within role rules	Normal approved jobs and reminders
FREEZING	New writes blocked; inflight work drains	Transition bookkeeping only
READ_ONLY / STANDBY	Blocked	No business polling, notification replay or reminder scans
RESTORING / VERIFYING	Blocked	Only the authorized transfer and verification job
QUARANTINED	Blocked	Operator diagnostics; no team activity

Planned handover

Tell staff the switch window. Check unsynced device work and open calls. Freeze the source, drain accepted writes and outbound jobs, record the final head, export and verify the saved archive. Import into the target, verify counts/files and representative staff views, then promote it and distribute its URL. Re-login and phone pairing are expected steps. Keep the old source read-only.

The promotion records the accepted parent, new write epoch, operator and time. Every business mutation and background worker checks the current local mode/epoch. Cached staff tokens cannot bypass a freeze. Scheduled reports, chat delivery, dialer dispatch and automated cleanup are included in the gate.

Disaster recovery

If the source account is already inaccessible, use the newest verified offline archive with the independent target credentials and offline key. Work committed after that archive's cutoff may be lost. Publish the known cutoff to the manager before promotion.

When the old instance returns, quarantine it before allowing staff writes. Compare lineage and reconcile any unsynced branch deliberately. A restored old instance must not immediately resume scheduled calls or message delivery.

Distributed limit

Without a shared online coordinator, one isolated host cannot prove that another independently administered host is read-only. The procedure and local mode gates prevent ordinary accidental dual use; lineage conflict checks detect divergence later. They do not constitute a global consensus service. Never claim an absolute dual-writer guarantee that the architecture does not provide.

Manager backup reminder: morning, evening, 72 hours

Planned schedule

Rule	Default behavior
Office timezone	Asia/Kolkata; store canonical timestamps in UTC
Morning prompt	10:30 IST, configurable
Postpone to evening	19:30 IST, bounded by the hard deadline
Postpone to tomorrow	Next 10:30 IST, bounded by the hard deadline
Hard deadline	72 hours after the last verified backup cutoff
No initial backup	Deadline is 72 hours from instance activation/bootstrap

Backup freshness is shared among authorized managers for the dataset on that active instance. One verified current backup satisfies the reminder instead of forcing every manager to read the entire database again. Imported old data uses its real saved cutoff, not the current import time, to avoid making an old snapshot look fresh.

After 72 hours, the manager sees a required backup step with no Postpone action. Backup/recovery, logout and connection help remain reachable. Other employees can continue ordinary CRM work; the rule does not silently lock the entire team out or start an expensive export without the manager's password and PIN.

Verified means saved

Starting a job or returning HTTP 200 does not reset the clock. A supported streaming save must close the file and verify its hash. A fallback browser download must be read back and checked. Cancelled downloads, incomplete files and failed verification do not count.

Persist server-side policy, last verified cutoff and snooze deadline. On login and focus, use a server-derived clock offset and lightweight existing status metadata. Browser timers schedule prompts locally; use existing event messages for job status. Do not query all tables every minute just to decide whether to show a reminder.

Standby instances do not wake employees, run business reminder scans or poll old candidates. On promotion, compute due work from the restored dates and acknowledgements with delivery deduplication. Current time may create newly overdue items; saved schedules must not disappear.

Egress, storage growth and recoverability

Reduce avoidable transfer

Keep existing CC873 request/cache reductions. Standby mode suppresses staff dataset polling, reminder scans and delivery jobs. Share one authorized export job among repeated clicks rather than generating parallel full snapshots. Display size/byte estimates, use compression and deduplicate identical file content.

After the core release, refreshing a verified local base archive can fetch only committed changes and new files while producing a fresh self-contained full archive. It reduces repeat transfer; it does not make new data or file downloads free.

Provider usage belongs to each environment and billing period. It is not business state to restore. Importing a backup does not reset a quota or reduce usage already charged. Record measured database bytes, Storage bytes and backend transfer separately; a percentage depends on the actual plan and allowance.

Control database growth honestly

Do not store each day's archive or a full shadow database in business tables. Keep compact revision metadata, tombstones and essential audit headers. Stage restore payloads privately with an expiry and reference-safe cleanup. Deduplicate only truly identical files, preserving all active references and required historical files.

Never delete thirty-day-old business activity merely to make the backup smaller. Do not erase call history, deleted-candidate reasons, interview attempts or attendance to save transfer. Long-term retention and legal requirements need a deliberate business policy; they are not inferred here.

Resource checks before a transfer

Check free DB space and write-ahead-log headroom, private temporary disk, object bucket capacity, largest file, browser supported size and service timeouts. A restore may need substantial transient space even when it eventually upserts existing rows. Reject unsafe jobs with an actionable capacity report instead of starting and corrupting availability.

Independent ownership checklist

Use separately controlled account credentials, MFA/recovery methods and offline recovery notes for the three environments. A backup kept only inside the same compromised mailbox does not solve the stated failure. Retain at least two independently stored verified archives and the private key separately from them. These account changes are an operator runbook; none were performed during planning.

Screen design and user flow

Add Project Multiple CRM under the manager administration area. Reuse the application's current theme and compact components. Show three equally sized instance cards for First, Second and Third with configured label, local/remote designation, known mode and last recorded handover. Do not constantly call all three hosts for a live badge.

Main actions

Action	Expected UI
Download whole CRM	Fresh password + PIN; coverage/size preflight; progress; saved-file verification
Upload whole CRM	Local archive and recovery key; validate/stage; explicit diff preview
Commit restore	Fresh authorization; target freeze; apply; verify; then enable promotion
Refresh local backup	Later phase; select previous archive and key; produce a new full archive
Change PIN / recovery settings	Reauthentication; current policy; key fingerprint and recovery instructions
Prepare a switch	Clear source freeze state, final cutoff and handover checklist

Keep the original candidate-only workbook tool labelled separately. Do not place a destructive reset beside normal backup as an ordinary primary action. A manager always sees the target instance, source dataset, cutoff and whether the file is complete before commit.

Preview content

Show total and per-family inserted, updated, unchanged, deleted-state/tombstone and conflicting entities; target-only rows; file counts/bytes; schema and lineage compatibility; local runtime exceptions; and the verified pre-restore rollback checkpoint. Conflicts are selectable for inspection but commit stays disabled until the plan is valid.

Progress identifies current stage and useful counts. Cancellation before commit is safe. Once commit begins, the UI explains that cancellation stops activation and follows recovery handling; it cannot promise to undo a database commit by closing the tab.

Accessibility and continuity

Use readable labels, keyboard focus, clear errors and reduced-motion support. A bounded animation or status transition is acceptable; large perpetual animation must not hide job state. Keep status on refresh using a job ID, without storing PINs, passwords or private keys in localStorage.

Server contracts to freeze before coding

Endpoint names below are a proposed contract. They are not deployed routes. All writes are manager-only and mode-aware, with structured validation and request limits appropriate to the stage.

Endpoint group under /api/multiple-crm	Responsibility
GET /status	Local mode, policy, head, backup freshness and compact job status
POST /settings/pin	Set/change the local PIN after reauthentication
POST /backup/grant	Password/PIN verification and operation-bound grant
POST /backup/jobs	Preflight and create one full or refresh export job
GET /backup/jobs/:id/stream	Authorized bounded download stream
POST /backup/jobs/:id/verify	Register verified saved-file receipt
POST /restore/jobs	Inspect header and create private staging job
PUT /restore/jobs/:id/chunks/:index	Authenticated bounded chunk transfer; idempotent by hash
POST /restore/jobs/:id/preview	Compute and bind a validated import plan
POST /restore/jobs/:id/commit	Fresh grant, target-head check and transactional application
POST /restore/jobs/:id/verify	Record post-commit logical/file checks
POST /restore/jobs/:id/rollback	Apply a validated undo procedure while read-only
POST /mode/freeze or /mode/promote	Recorded operating-mode transition with epoch check

Job statuses are QUEUED, READING/STAGING, VALIDATING, PREVIEW_READY, COMMITTING, VERIFYING, READY, COMPLETE, FAILED, CANCELLED or EXPIRED as appropriate. Define allowed transitions explicitly; only committed and verified jobs can reach a successful restore state. Export completion additionally requires saved-file verification.

Errors carry a stable code, safe message, job ID and retryability: AUTH_REQUIRED, PIN_COOLDOWN, INCOMPLETE_ASSETS, SCHEMA_MISMATCH, DATASET_MISMATCH, BRANCH_CONFLICT, PREVIEW_STALE, CAPACITY_LOW or CHECKSUM_FAILED. Avoid embedding decrypted records or secrets in errors.

GET status and progress are read-only. No GET starts an export, commits a restore or reads every business table. SSE/status traffic contains metadata only and stops on standby or expired jobs.

Code and SQL integration map

Existing area	Planned integration
backupController.js	Split full workflow into grant/job/preview/commit steps; retain scope separation
backupCodec.js	Versioned streaming envelope; legacy read adapter; authenticated chunk/trailer tests
backupRestoreCompat.js	Typed identity/reference adapter; remove fuzzy matching from v2
store.js and write entry points	Transaction-bound revisions, mode/epoch gates, catalog drift checks
routes/index.js and app.js	Capability/CSRF gates and bounded stream/chunk parsers
DataBackupPage.jsx	Manager instance cards, verification, diff preview and job recovery
Layout.jsx and reminder integration	Manager deadline prompt and quiet standby behavior
Supabase migrations	Safe identity backfill, control schema, revision guards and ID generator rules
Android and local test launchers	Target URL/re-pair handover instructions; smoke checks

Keep a small number of cohesive modules. This map does not require creating one file per endpoint. The earlier delivery constraint remains frontend fewer than 98 files and all other GitHub files combined fewer than 98. Measure the final package; do not satisfy the limit by omitting runtime source, migrations or manifests.

Proposed final five-part release

1. GITHUB frontend: one frontend folder with all required sources and manifest.
2. GITHUB backend/root: one backend folder and required deployment/root files, without unnecessary nested copies.
3. SUPABASE: First/Second/Third folders, safe SQL and matching long-script parts, with explicit order.
4. ANDROID: Android Studio project and verified build/open instructions.
5. LOCAL_TEST, TEMPLATES and README/Architecture: local launcher, existing upload templates, updated historical work log and architecture PDFs.

All PDFs travel in README/Architecture, including in this planning ZIP; historical work records remain under its HISTORY folder. This documentation pack does not replace any of those application parts.

Build constraints

Use the inspected Node 24 and React stack initially. Prefer native streams, crypto and zlib with browser Web Crypto. New dependencies, if needed for bounded browser file handling or streaming, require version/security review and a documented reason. Lock resolved dependencies and record exact build tools at release time.

Implementation phases and exit gates

Phase	Work	Exit gate
0	Live-like fixture inventory, complete table/key/bucket catalog; reproduce baseline gaps	No unknown durable object; agreed coverage and compatibility fixtures
1	Canonical identity, lineage, revision guard, mode gate; three safe SQL bootstraps	Reruns preserve data/UIDs; direct SQL writes are tracked; standby blocks all writers
2	Manager password/PIN, grants, offline recovery key and trust setup	Role, replay, key-loss and cryptographic negative tests pass
3	Full streaming database/files export and saved-file proof	Large archive round-trip; no missing owned bytes or unbounded allocation
4	Staged restore, diff/conflicts, DB commit, ID advance, undo and activation	100-to-200 round trip and failure-injection suite pass
5	Daily/72-hour reminder, switch UX, quiet standbys and mobile handover	Time-boundary and no-background-poll checks pass
6	Refresh existing local archive from committed deltas	New file restores alone; delta/base integrity tests pass
7	Isolated A/B/C integration, measured resources/egress, packaging and handover	Evidence report, release hashes, PDFs and five ZIP parts verified

Do not implement the low-egress refresh optimization before a reliable full archive exists. Do not enable real staff switching after a UI-only demonstration. Each phase updates the work record with source files, schema version, executed tests, failures, fixes and remaining limitations.

Rollout order

Build and test in disposable isolated fixtures. Produce a verified backup of the real First dataset before any additive migration. Apply the safe First upgrade, then bootstrap empty Second and Third with independent local credentials. Register the dataset/public recovery key and source-trust fingerprints deliberately. Run a restore rehearsal before a production handover.

Keep old runtime paths feature-gated until the new workflow passes. Disable legacy full-download/restore bypasses when enabling the new gates. Never mark a migration complete merely because the script printed success; check data, versions, keys and sequence behavior.

No execution estimate is asserted here. Actual size, provider limits and browser behavior determine resource choices. A failed exit gate keeps the feature in its prior safe mode and is recorded, rather than counted as a pass.

Documentation and handover discipline

Required update on every future change

Record the user's requested outcome, final implemented behavior, changed files/functions and migrations, reason for the change, actual verification evidence and outstanding work. Preserve abandoned approaches only when they explain a remaining tradeoff or compatibility decision.

Update the relevant topic PDF and editable Markdown in the same release. Keep the old historical PDF as an unchanged source and add a dated work-record entry. A new chat or developer should read the current status ledger first, rather than assume every design paragraph is implemented.

Status vocabulary

State	Meaning
PLANNED	Design exists; feature is not implemented
IMPLEMENTED_NOT_VERIFIED	Source exists; acceptance evidence is missing
VERIFIED_LOCAL	Named local fixture tests passed, with scope recorded
VERIFIED_STAGING	Isolated hosted integration passed, with sizes and versions recorded
VERIFIED_PRODUCTION	Explicitly observed production operation, with safe audit evidence
BLOCKED	Named dependency or failing test prevents progress

Current Multiple CRM work is PLANNED across all implementation phases. Document generation, source audit and consistency checks are complete planning activities only. No hosted restore, nine-hour multi-user exercise, real provider measurement or live disaster recovery drill was performed.

Instructions for the next developer or ChatGPT

Read 00_START_HERE.md and these six topics, then inspect the actual CC873 source and original requirements. Verify hashes and resolve any later source version before editing. Start at phase 0; do not run destructive SQL. Do not claim the new security or migration features already exist because this pack explains them.

After each phase, update IMPLEMENTATION_STATUS.json and the dated work record with evidence paths. Keep independent offline copies of the recovery kit and archives. Generated documentation must not contain real service credentials, passwords, PINs or private keys.

Planning handover boundary

This pack is concrete enough to review and begin implementation, while retaining honest implementation gates. The next action is construction of the catalog/identity foundation, not a live account reset or a silent deployment. Final user delivery of the working system follows its own cross-check and five-part packaging step.

Acceptance plan: data identity and complete coverage

All tests listed in this document are NOT RUN for Project Multiple CRM. They define release gates. Each result must identify the fixture, code/schema version, exact command or procedure, assertions, evidence and actual outcome. Historical CC873 checks are separate.

ID	Scenario	Required result
D01	Export/import 100 candidates twice	First apply correct; second is a no-op with no duplicates
D02	First 100 to Second 200 to First	Original IDs match; new 100 and changed old rows retained
D03	Ten attempts, same person and same client	Ten distinct attempts and correct client counts remain
D04	Soft delete, recycle reason and tombstone	No disappearance and no resurrection on older copy
D05	Older archive over a newer head	Stale import blocked before active writes
D06	Both instances edited after common baseline	Branch conflict shown; no silent winner
D07	Same C101 for two different UUIDs	Collision blocked; explicit reconciliation required
D08	Different dataset UUID, similar row names	Rejected as ordinary continuation
D09	Same username, different employee identity	No incorrect owner/team remapping
D10	New target changes after preview	PREVIEW_STALE; no old-plan commit
D11	Sequence 500 with lower imported live IDs	Next generated ID remains above safe high-water
D12	Deleted/reserved highest candidate ID	No ID reuse after restore or failed transaction
D13	All 80 table policies and unknown durable table	All catalogued families covered; unknown blocks Complete
D14	Mixed settings keys and credentials	Business presets portable; local secrets unchanged
D15	Concurrent commits during export/refresh	One consistent cutoff; late commits not skipped

For D02, verify more than candidate counts: ownership, notes, attempts, client history, drafts, tasks, files, roster and status changes in both directions. Canonical row hashes and per-family counts catch fields that a total count misses.

Use representative legacy schemas, partial migration states and all three SQL folders. Test first run, rerun and both-part ordering. Backfilled UUIDs and displayed IDs must remain stable. Existing conflicting IDs must be diagnosed before enforcing constraints, with no partial destructive cleanup.

Acceptance plan: files and security

ID	Scenario	Required result
F01	Recording in Supabase Storage only	Real bytes restore and playback; old host not required
F02	Resume, Word/PDF and database byte content	Correct hashes, filenames, access and preview
F03	File larger than old 16 MiB cap; total over 32 MiB	Bounded transfer succeeds within new tested policy
F04	Same bytes used by multiple rows	One archive object; all references remain valid
F05	Missing required object or changed byte	No Complete result or restore commit
F06	Owned orphan object and external link	Inventory handles orphan; external dependency declared
F07	Malicious path/length/member metadata	Reject safely before filesystem or resource misuse
F08	Target URL/bucket differs from source	Typed references rebound; text history unchanged
S01	Recruiter/TL, forged role, impersonated session	All direct full-export/import APIs reject
S02	Wrong password/PIN and repeated attempts	No grant; tested cooldown and safe recovery path
S03	Replayed/expired grant or changed archive	Rejected; commit bound to actor/session/job/hash
S04	Source Gmail unavailable	Restore succeeds with target access, archive and offline key
S05	Wrong/missing private recovery key	Decryption fails; no fallback that weakens archive secrecy
S06	Altered/reordered/truncated encrypted chunks	Authentication failure; no trusted partial content
S07	Unknown source signer/key replacement	Trust mismatch; explicit review before import
S08	Legacy full route bypass attempt	Same password/PIN/capability gate applies
S09	Logs, localStorage and staging access review	No secrets, public private payloads or unbounded PII logs

Security checks include actual HTTP calls, not just hidden UI buttons. Exercise session expiry during long jobs and reauthentication before commit. Negative cryptographic test vectors must cross Node 24 and the supported browser implementation, including nonce uniqueness, final trailer and bounded decompression.

Acceptance plan: failures, operations and cost

ID	Scenario	Required result
O01	Storage upload or DB constraint failure	Old active state survives; staged garbage safely isolated
O02	Process restart mid-upload	Same archive resumes or expires safely; no mixed job content
O03	Post-commit file/UI verification failure	Target stays read-only; undo procedure tested
O04	Cancelled download or failed local readback	Reminder remains due; old verified file stays valid
O05	Planned freeze during active writes/calls	Inflight work handled; all later writers/workers gated
O06	Old source returns after target promotion	Old host quarantined; no external delivery replay
O07	Historic presence, breaks and paused dialer	History retained; live runtime starts deliberately
O08	Phone unsynced draft/outbox; target re-pair	Unsent work reported/drained; tokens not cloned
O09	10:30, 19:30 and exact 72-hour boundary	Correct manager-only prompt and bounded postpone
O10	Wrong browser clock, offline login, missed deadline	Server-based due state survives refresh and reconnect
O11	Standby left open all day	No business polling or reminder-table scanning
O12	Simulated large dataset and many files	Bounded RAM; reported DB/WAL/temp headroom respected
O13	Repeated backup clicks and two managers	Shared job/receipt; no accidental duplicate full reads
O14	Refresh local base with updates/deletes/files	New archive restores alone without its former base
O15	Invalid base signature or late-commit delta	Refresh refused or falls back; no skipped data
O16	Expired job cleanup and retained business history	Temporary bytes removed; referenced/history bytes kept
O17	Full A/B/C round trip with staff role views	Real roster access, cards, tasks and due schedules correct
O18	Measured egress and repeated daily workload	Publish actual bytes/limits; no invented percentage pass

For hosted verification, record provider plan, dataset size, object bytes, process peak RSS, wall time and upload/download failures. Use disposable instances for fault injection. A real multi-user endurance test remains a separate named exercise; do not claim nine hours of reliability from fast local fixtures.

Check final frontend and other GitHub file counts, Android project open/build, local test browser flow, SQL part equivalence, ZIP CRCs and that every package carries the current documentation. Those packaging checks do not replace data-recovery tests.

Operator runbook: daily backup and planned switch

Daily backup

1. Open Project Multiple CRM as a manager on the ACTIVE instance. Confirm dataset, current head, local mode and last verified cutoff.
2. Choose Download whole CRM, or the later verified local-refresh flow. Enter current password and PIN. Review coverage and estimated resource use.
3. Save the archive to an independent location, complete local verification and check the receipt shows the expected cutoff. Keep the offline key separately.
4. Retain another verified independent copy. Periodically restore to an isolated standby and check representative records/files. A saved filename alone is not recovery proof.

Planned First-to-Second switch

1. Confirm Second has compatible code/schema, capacity, independent credentials, trusted source key and recovery public key. Do not seed competing business rows.
2. Tell staff to finish work and check phone outboxes. Freeze First and verify writes/workers have drained. Record its final accepted head.
3. Export and verify the frozen cutoff. On Second, upload/decrypt, stage, inspect the diff and resolve all blockers.
4. Commit using fresh authorization. Verify counts, candidate attempts, ownership, tasks/reminders and actual file access. Keep Second read-only until verification passes.
5. Promote Second, give staff its URL, sign in and pair phones. Keep First read-only. Record the new epoch and the manager's handover.
6. Repeat the same procedure when returning the newer Second dataset to First. Check that First is its ancestor; investigate target-only changes before any commit.

Before any destructive source action

No export or restore automatically destroys a source. If the operator later intends to retire an account, first verify independent target access and an offline restore drill with the newest archive. The planning task does not authorize deleting projects or Gmail accounts.

Record a short switch report: source/target IDs, backup UUID/cutoff, before/after heads, logical counts, file coverage, exceptions, verification evidence and promotion time. Store no password, PIN or private key in the report.

Operator runbook: disaster recovery and rollback

Source account unavailable

1. Use independent target login/recovery-owner access. Do not depend on password-reset email reaching the inaccessible original mailbox.
2. Locate the newest verified archive and the offline private key. Read its cutoff aloud to the operator; later unsaved work may be absent.
3. Install the matching approved release/schema from the recovery kit if required. Verify trusted key fingerprints from independent notes.
4. Stage, preview and commit on the read-only target. Verify business data and actual files, then promote. Staff sign in anew and devices re-pair.
5. If the old source returns, quarantine it before staff activity. Compare branch heads and collect any unsynced work for explicit reconciliation.

Restore fails before commit

Keep the target in its prior safe state. Inspect the job's error code and coverage report. Correct missing capacity, incompatible schema or file access without changing business rows. Retry the same immutable archive only under the job's hash and lease rules. Remove expired unreferenced staging after the retention window.

Restore fails after commit

Keep the target read-only and block dispatch workers. Preserve the job evidence and new files with live references. Diagnose whether verification can be completed safely. If rollback is needed, use the encrypted pre-restore undo checkpoint and verify the resulting head, IDs, files and logical counts before reactivation.

Sequence values may have advanced despite a rollback; preserve safe high-water values. A gap is expected and is not repaired by decrementing the generator. No candidate ID or attempt ID is reused to make the list look continuous.

Key or trust failure

If the offline key is missing, do not reset the PIN and pretend it decrypts the file. Find an independent key copy or use a still-accessible source to create a newly keyed archive. If a source signing identity changed unexpectedly, stop normal import and review independent records; do not auto-trust the uploaded replacement.

Completion evidence

The operator report distinguishes restored business state from intentional live-runtime exceptions and any unavailable device-only edits. It contains the exact recovery point and limitations. It must not claim a lossless disaster recovery when the latest verified backup is older than the source's final work.

Work record and official source register

Dated work entry - 20 October 2026

Request: design Project Multiple CRM before coding, account for duplicate/conflict safety and files, retain a detailed architecture handover, then report the planning checkpoint.

Completed in this planning operation: read the local original requirements; reviewed the eight baseline backup/integration files and relevant storage/recording behavior; documented eleven baseline gaps; designed dataset identity, one-writer handover, complete archive coverage, manager gates, offline-key recovery, restore/undo stages and reminder rules; classified all 80 existing table names; defined the build phases and 50 named acceptance cases.

Produced six topic PDFs, editable counterparts, the table catalog, status/requirement ledgers and this dated work entry. Preserved the prior CC873 PDF unchanged. Final planning QA checks document rendering, table/requirement coverage, unchanged source hashes, internal package paths and ZIP integrity.

Not done: new CRM code, new SQL, PIN or key setup, live backup/restore, hosting deployment, real egress measurement, Android handover test or multi-user endurance test. Those remain the gated implementation work in document 05.

Primary sources checked during planning

Source	Fact used
Supabase: Database Backups	SQL backups do not contain actual Storage object bytes
Supabase: Manage Egress Usage	Data and file transfer contributes to provider egress
Supabase: Billing on Supabase	Free-project eligibility is account/org policy, not a three-email guarantee
PostgreSQL: Transaction Isolation	Repeatable-read snapshots and special sequence behavior
PostgreSQL: Sequence Functions	Sequence advancement can survive rollback; gaps are normal
Node.js 24: Crypto	Native primitives proposed for the versioned archive protocol

The source URLs and access date are in SOURCES/OFFICIAL_REFERENCES.md. Architecture decisions, such as mode gates and lineage policy, are proposed project designs; they are not claims that the provider implements them.

Resume note

Start the next work session at phase 0 with the five CC873 application parts and this pack. Confirm the baseline version first. Keep implementation status and test results honest, and append work evidence on each later change. Planning readiness does not remove the requirement to cross-check the constructed system before final release.

Your CC875 update

Dashboard views and reminders added to the recovered CC874 Project Multiple CRM source.

Area	Result
Switch dashboard	Manager: any TL/recruiter. TL: authorized team recruiters. Recruiter: own view only.
My dashboard	Own profiles, submissions, interviews, tasks, follow-ups and Pipeline data. Overview remains available.
Organized Reminders	Categories with counts, search, Open, Snooze and Dismiss. Existing reminders remain saved.
Popup settings	Manager team toggle, default 10-second auto-hide, 1-300-second duration and Stay open.
CC874 recovery	Whole backup, encrypted restore, local-base refresh, three independent instance slots and offline recovery preserved.
Packaging	Five separate ZIPs. Frontend 97 files; all other GitHub source together 97.

What was checked

50 local functional, database and browser checks passed in this run. Checks use isolated fixture accounts and databases; no production Supabase or Render data was changed.

Current work is appended after the retained CC873 work record and the original 38-page approved architecture plan. Those earlier pages are historical context. This eight-page CC875 section is the current implementation and verification record.

One dashboard, across your work

Choose a view once; the business pages use it together.

Signed-in role	Available views
Manager / Admin	My dashboard, all dashboards overview, any active TL or recruiter.
TL	My dashboard, own team overview, authorized recruiters in that team.
Recruiter	Own existing dashboard; no switch control.

Behavior

• Use Switch dashboard in the top bar. A banner identifies the active personal view. My dashboard is own-only; Return to overview restores the existing consolidated view.

• Selection survives navigation and refresh for the current signed-in account. Candidate lists, submissions, interviews, tasks, Pipeline, follow-ups, search and drafts use the selected identity.

• Viewing another employee is read-only. Your login, attendance, settings and administrative identity remain yours. No password or employee login is borrowed.

• The backend checks the target against your role and team. A moved or no-longer-authorized recruiter is rejected after the existing access cache is invalidated.

Cache and live updates

React reads, shared browser reads and reminder snapshots include the selected view. Late replies from an earlier view cannot repopulate the new one, including a rapid A-to-B-to-A switch. Temporary instant profile snapshots are cleared on a view change. Existing event updates invalidate the current view; manual refresh remains available.

Quick check

Switch to a recruiter, compare the visible candidate row and card counts, open another business section, then refresh. Use My dashboard and verify only your work remains. Return to overview to restore the team/all view.

Reminders, organized

A compact popup for attention; the complete queue for review.

Category	Items to review
Follow-ups	Calls and follow-ups linked directly to the candidate.
Interviews & feedback	Interview and feedback reminders.
Submissions & approvals	Pending submission and details/approval work.
Tasks	Due task work.
Reports / Updates / Other	Report reminders, notifications and remaining reminder types.

Category counts and search use the loaded queue. They do not issue a new database request for each click. Refresh Now explicitly checks current reminder data. Open keeps the candidate link, and snooze/dismiss remain deliberate actions.

Settings > Popups & reminders

• Show my team’s popups: on by default for managers; off requests only the manager’s own work reminders. A deliberately selected staff dashboard still shows that selected person’s work.

• Hide popups automatically: on by default, after 10 seconds. Choose 1-300 seconds, or turn it off to keep popups visible until closed.

• Auto-hide only hides the current visual reminder batch until its normal repeat window. It does not delete a reminder, mark work complete, approve attendance or change a saved candidate.

• The attendance-decision information prompt follows the timer too. Office/access locks and required recovery confirmation screens remain visible.

The compact staff reminder presenter is also used for managers. The redundant manager center and large in-CRM task presenter do not create a second browser popup when the compact owner is active.

Lightweight daily use

This update avoids adding a new periodic data-fetch loop.

Action	Network/storage behavior
Open dashboard chooser	One small employee identity lookup on opening; backend roster cache is bounded.
Choose category / search	Filters the existing reminder list locally.
Auto-hide / Stay open	Browser timers; no reminder delete or approval request.
Change popup preferences	Small personal browser preference state; no new cloud table or file.
Switch employee view	Loads the selected view using existing business endpoints and permissions.
Recovery workspace	Business fetches and chat/approval/secure-link streams stay paused on the backup page.

Important findings fixed during this run

• An older cross-tab read cache did not include dashboard scope and could reuse a recent empty/team reminder response. It now isolates the view and popup audience, and honors explicit manual refresh.

• An inline early event broker overrode the later broker, missing recovery-mode gating and newer access-change events. Both now use the current behavior.

• A popup could disappear then immediately reveal another old reminder. Auto-hide now gives the loaded due queue a quiet repeat window while retaining every reminder in the list.

What remains unmeasured

Actual provider-billed egress was not measured here. Opening new profiles, fetching changed rows, backups and downloading files necessarily transfers bytes. A fixed daily 2% ceiling or zero egress cannot be guaranteed from local code tests. Measure real usage with all 20 users before claiming a production budget.

Project Multiple CRM retained

Whole recovery remains separate from candidate-only Excel export.

Backup and refreshed backup

Whole capture briefly freezes source writes, reads a consistent snapshot and owned files, then returns to the prior mode. Stable row identities, deletions, all interview attempts, source files and matching SQL are included. Owned file bytes are hashed and deduplicated. The archive is encrypted for the registered offline recovery key.

Refresh using a saved backup validates the old archive on the browser, uploads authenticated packets into encrypted temporary staging, and prepares an encrypted local base index. Unchanged rows and files can be reused. The new archive remains complete and independent of the old base.

Restore sequence

Register the matching public recovery key and trusted source signing key on an independent target. Verify the saved archive locally, stage packets, inspect the signed preview, commit the exact preview and verify rows/files. The target stays frozen until deliberately promoted. Undo is limited to the pre-promotion unchanged target state.

Security and correctness kept

• Manager password plus four-digit PIN; action-bound one-use grants; persisted wrong-PIN cooldown.

• Encrypted temporary packet and file staging; tampered staged bytes fail before candidate changes.

• Recovery owner remains independent of restored business users and survives auth/me and browser reload.

• Private Storage files use authenticated candidate/employee access links, range playback and current ownership checks. Embedded JSON file references are remapped too.

• Restore preserves accumulated attendance totals, but clears transient presence and prevents disconnected offline time from earning work credit.

The one-active-writer handover remains an operator-controlled procedure across independent providers; it is not a distributed cross-host lock. Freeze the previous source before activating another instance.

Install the five parts

Extract all five ZIPs into the same parent folder. They share CC875_FINAL.

Part	Contents
1 - GitHub frontend	DESKTOP_CRM/GITHUB/frontend: 97 source files.
2 - Backend + root	DESKTOP_CRM/GITHUB backend, scripts and root config: 97 files combined.
3 - Supabase	First keep-data upgrade; guarded empty Second/Third setup; full or two-part paths.
4 - Android	Retained CC873 / versionCode 278 source and Android Studio batch workflow.
5 - Local / templates / README	Built local browser preview, local accounts, Hot Lead templates, current guide and historical record.

Web deployment

Upload the contents of GITHUB to the repository root. Use Node.js 24, build with npm run render-build, start with npm start. Keep existing deployment credentials. The build regenerates SPA/assets and verifies source counts. Deploy frontend and backend together, then refresh the browser.

SQL choice

The new CC875 dashboard/reminder controls need no new tables. If the complete CC874 schema/control is already installed, no additional SQL is required for these controls. For an earlier existing CRM, First’s safe SQL preserves rows and existing instance slot/mode. Use the complete file OR the two parts in order. Second/Third bootstrap refuses populated databases. Historical reset files are reference-only and must not be used for an upgrade.

Android and local test

The Android source is preserved, not rebuilt. Its native version remains CC873 / 278; the web is CC875. Open the retained Android Studio batch file to build. The local launcher opens a separate local preview with explicit local accounts and no production Supabase connection. Hot Lead CSV/XLSX templates are unchanged.

Evidence and remaining checks

50 current local checks passed. The earlier release’s claimed 39 checks are not substituted for this run’s results.

Current suite	Passed
Streaming backup / restore / refresh	25
Fresh / rerunnable / split SQL	3
Dashboard role/access + Chromium UI	16
HTTP Storage + recovery browser	6
Total	50

Machine-readable names of every check are in README/TEST_RESULTS. Fresh checks include 100-to-200 profile round-trip, repeated interview attempts, encrypted file corruption rejection, branch conflict, rollback, UUID tracking, local-base independence, zero-byte/private files, role restrictions, current ownership, persisted dashboard, popup retention, category search and a 390px browser view.

Run in the real environment next

• Live Supabase egress: record a baseline and compare a full working day across all 20 accounts, including calls, updates, files and reminders.

• Nine-hour multi-user endurance: allocation changes, concurrent saves, network recovery and session behavior.

• Real Render deployment and provider restart/recovery: confirm credentials, schema and independent Storage behavior.

• Android Studio build and a real device test: Gradle/JDK/SDK, signing, pairing, calls and device-specific sharing.

No live reset, deployment, provider migration, production-user login or database restore was executed in this run. The included checklist groups the manual work by feature and role so it can be verified without changing logins.

Worklog and continuation

What was recovered, what changed, and what to check next.

Finding / request	Action taken
Use the earlier CC874 release	Recovered the retained CC874 work checkpoint and approved plan. Rebuilt documented final hardening and tested the recovered source. This is CC875, not a byte-for-byte copy of a missing old ZIP.
Scope missing or stale after switching	Added one backend-authorized view selection, scoped read caches and stale-response guards.
Manager must see staff reminders	Used consolidated reminder data and one compact popup. Added personal audience setting.
Popups stay forever / hide safely	Added configurable visual timers and retained the saved reminder queue.
Queue difficult to review	Added category counts, search and direct candidate actions.
Old backup-page traffic	Fixed the early broker and recovery layout so business streams/reads stay paused.
Android download metadata stale	Corrected the exposed native source version to CC873 / 278 while marking web CC875.

Carry-forward record

The original approved plan and CC873 work record remain ahead of this section. Historical files are retained under README/Architecture and README/HISTORY_CC873. Current implementation status, exact source checksums, package checks and the next-time checklist accompany this record.

No hidden completion claim

Local behavior is verified to the scope listed above. Provider egress, sustained live concurrency, real Render deployment and Android build/device behavior remain unverified until those checks run on the actual environment.